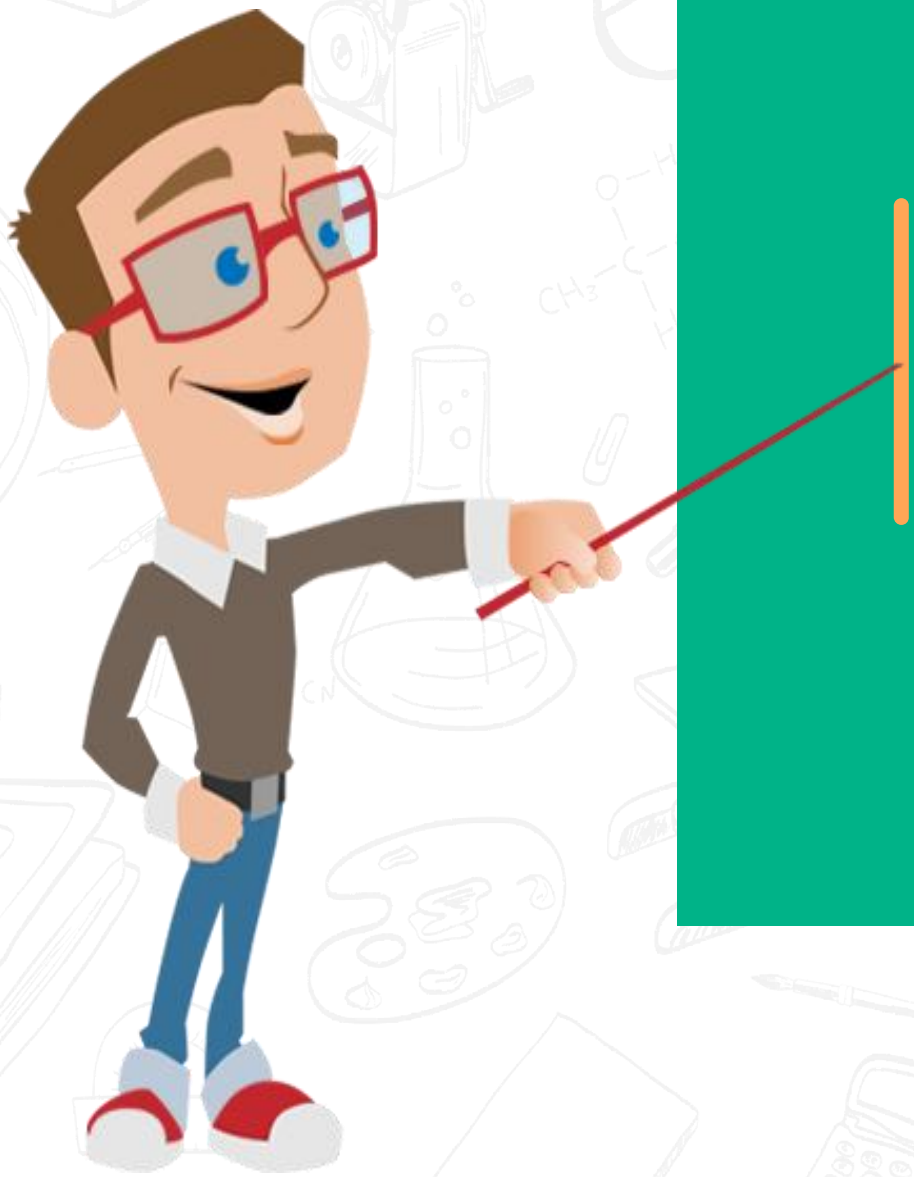


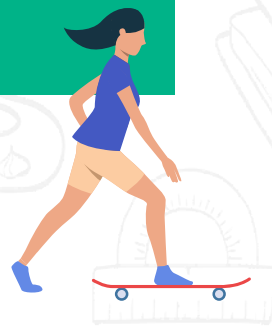
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 15-1

1 Xacro 模型优化

2 Gazebo 控制小车

3 自主导航

4 ~

Objectives

1、修改自己的Xacro模型，使其能在Gazebo中正常显示

2、能够使用teleop_twist_key在Gazebo中控制小车

3、运行导航模块中的navigation功能包



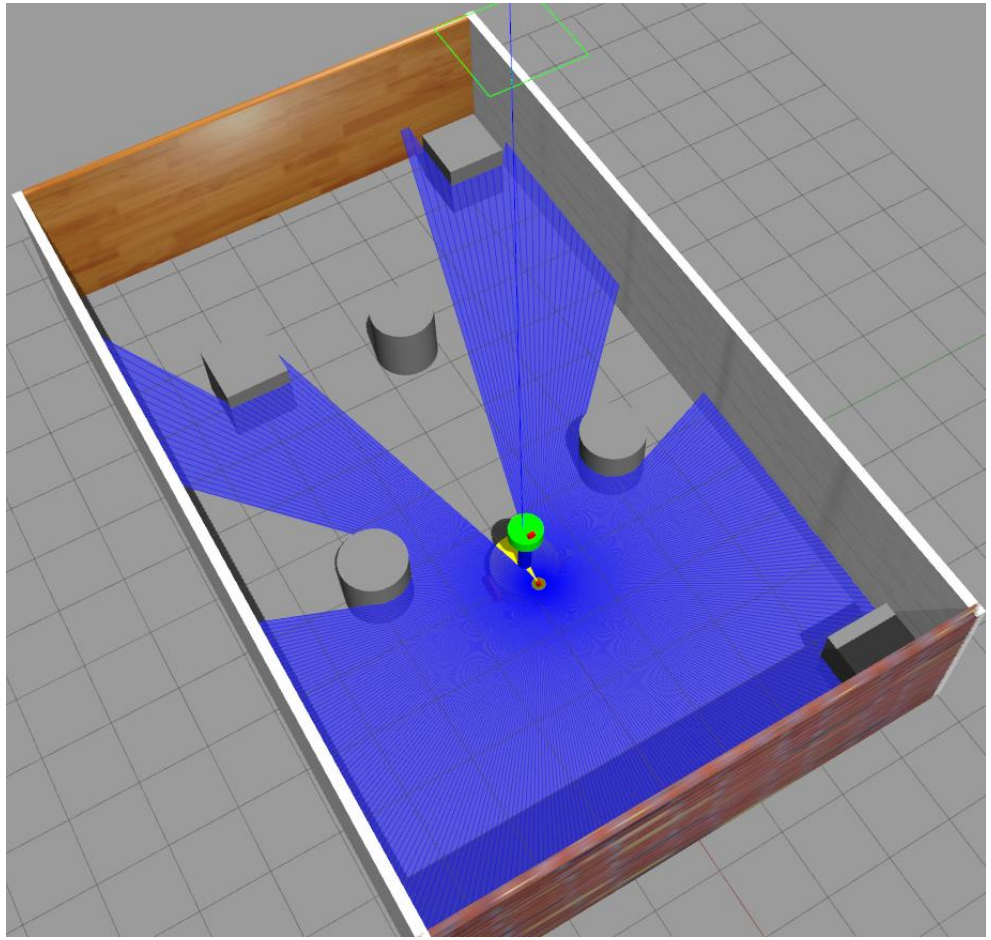


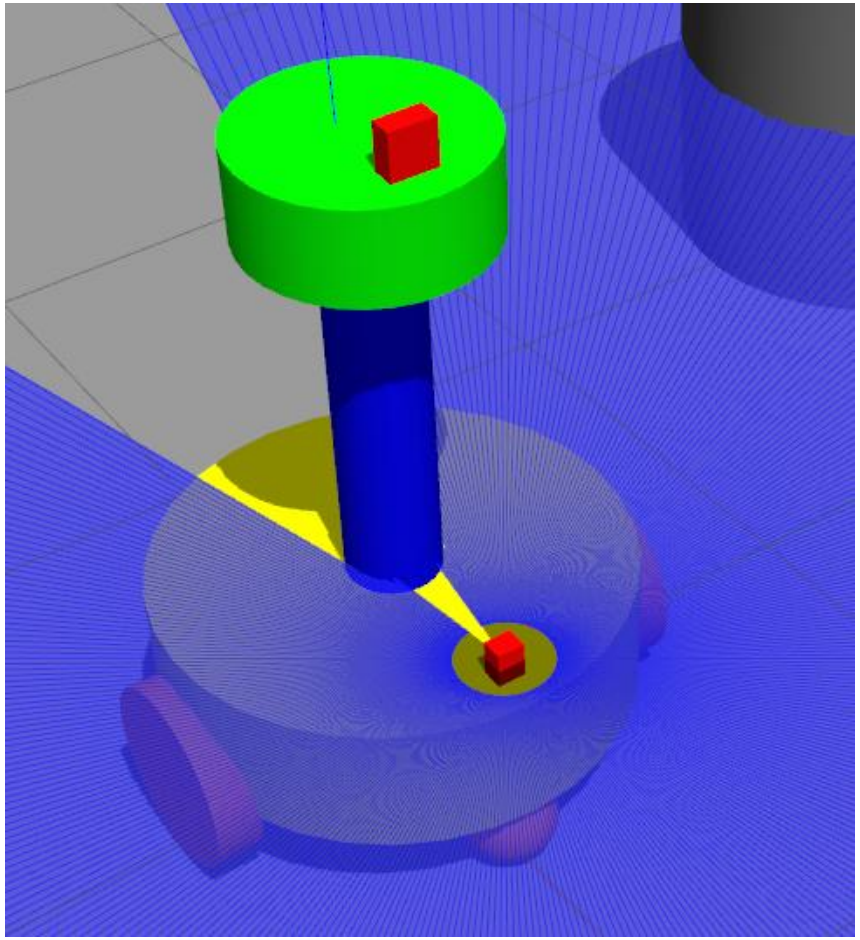
1

Xacro 模型优化

1.1 Xacro 模型回顾

1.1.1 myrobot_whole_func.xacro

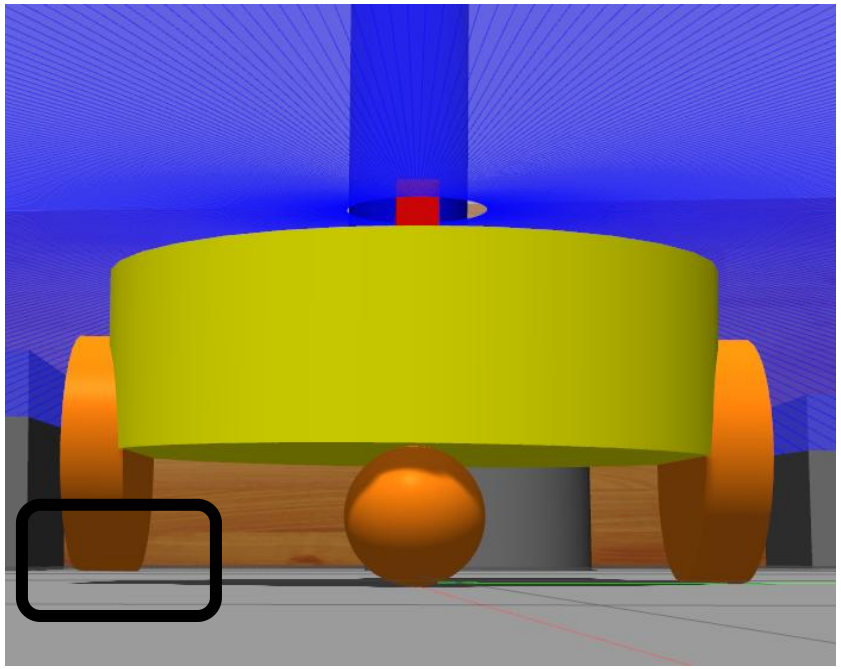




1.1 Xacro 模型回顾

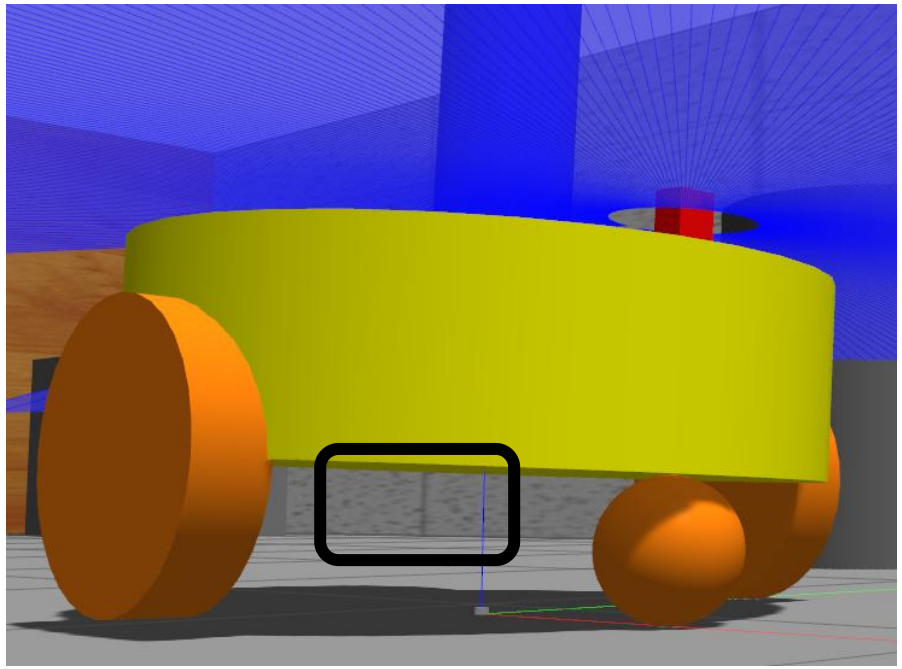
1.1.1 myrobot_whole_func.xacro

- 问题1：右轮有离地间隙



- 产生原因：base_link 和 root_link 的偏移量计算有误

- 问题2：模型运动时会后仰

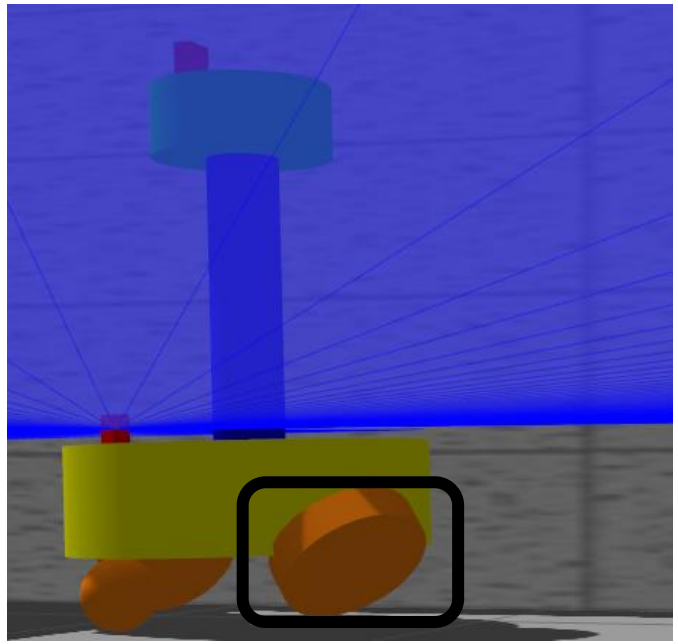


- 产生原因：缺少后支撑轮

1.1 Xacro 模型回顾

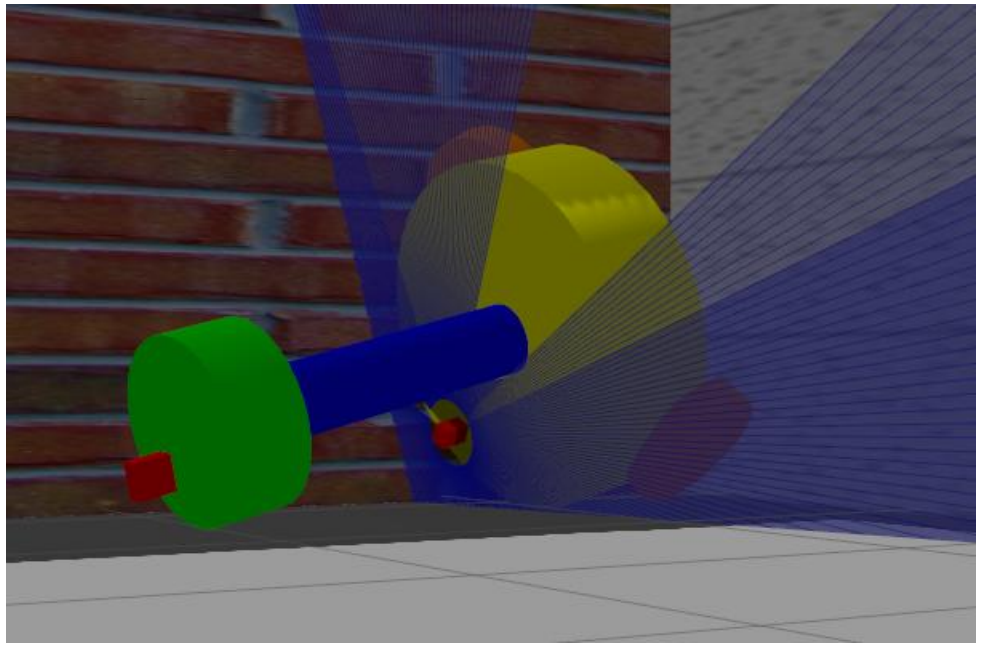
1.1.1 myrobot_whole_func.xacro

- 问题3：两个驱动轮无法绕 y 轴转动



- 产生原因：驱动轮未设置 y 轴自由度

- 问题4：模型倾倒



- 产生原因：(1) 模型 link 质量设置不合理；(2) root_link 设置了 inertial，增加不稳定性(较难)

1.2 Xacro 模型优化

1.2.1 myrobot_gazebo.xacro

- 问题1：右轮有离地间隙

```
<xacro:joint_info child_link="base_link" parent_link="root_link"
type="fixed" ox="0" oy="0" oz="${base_height / 2 + wheel_radius}"
ax="0" ay="0" az="0"/>
```

1.2 Xacro 模型优化

1.2.1 myrobot_gazebo.xacro

- 问题2：模型运动时会后仰

```
78  <xacro:macro name="support_wheel" params="support_wheel_name flag gazebo_color m r">
79      <link name="${support_wheel_name}_support_wheel_link">
80          <visual>
81              <geometry>
82                  <sphere radius="${support_wheel_radius}" />
83              </geometry>
84              <origin xyz="0 0 0" rpy="0 0 0" />
85              <material name="Voilet">
86                  <color rgba="0.933 0.51 0.933 0.8" />
87              </material>
88          </visual>
89          <collision>
90              <geometry>
91                  <sphere radius="${support_wheel_radius}" />
92              </geometry>
93              <origin xyz="0 0 0" rpy="0 0 0" />
94          </collision>
95          <xacro:sphere_inertial_matrix m="${m}" r="${r}" />
96      </link>
97      <gazebo reference="${support_wheel_name}_support_wheel_link">
98          <material>${gazebo_color}</material>
99      </gazebo>
100     <joint name="${support_wheel_name}_support_wheel_link2base_link" type="continuous">
101         <parent link="base_link"/>
102         <child link="${support_wheel_name}_support_wheel_link" />
103         <origin xyz="${flag * 0.8 * base_radius} 0 ${-(base_height / 2 + support_wheel_radius)}" rpy="0 0 0" />
104         <axis xyz="1 1 1" />
105     </joint>
106 </xacro:macro>
107
108 <xacro:support_wheel support_wheel_name="front"  flag="1"  gazebo_color ="Gazebo/Orange"  m="${support_wheel_mass}"  r="${support_wheel_radius}"/>
109 <xacro:support_wheel support_wheel_name="back"  flag="-1"  gazebo_color="Gazebo/Orange"  m="${support_wheel_mass}"  r="${support_wheel_radius}" />
```

1.2 Xacro 模型优化

1.2.1 myrobot_gazebo.xacro

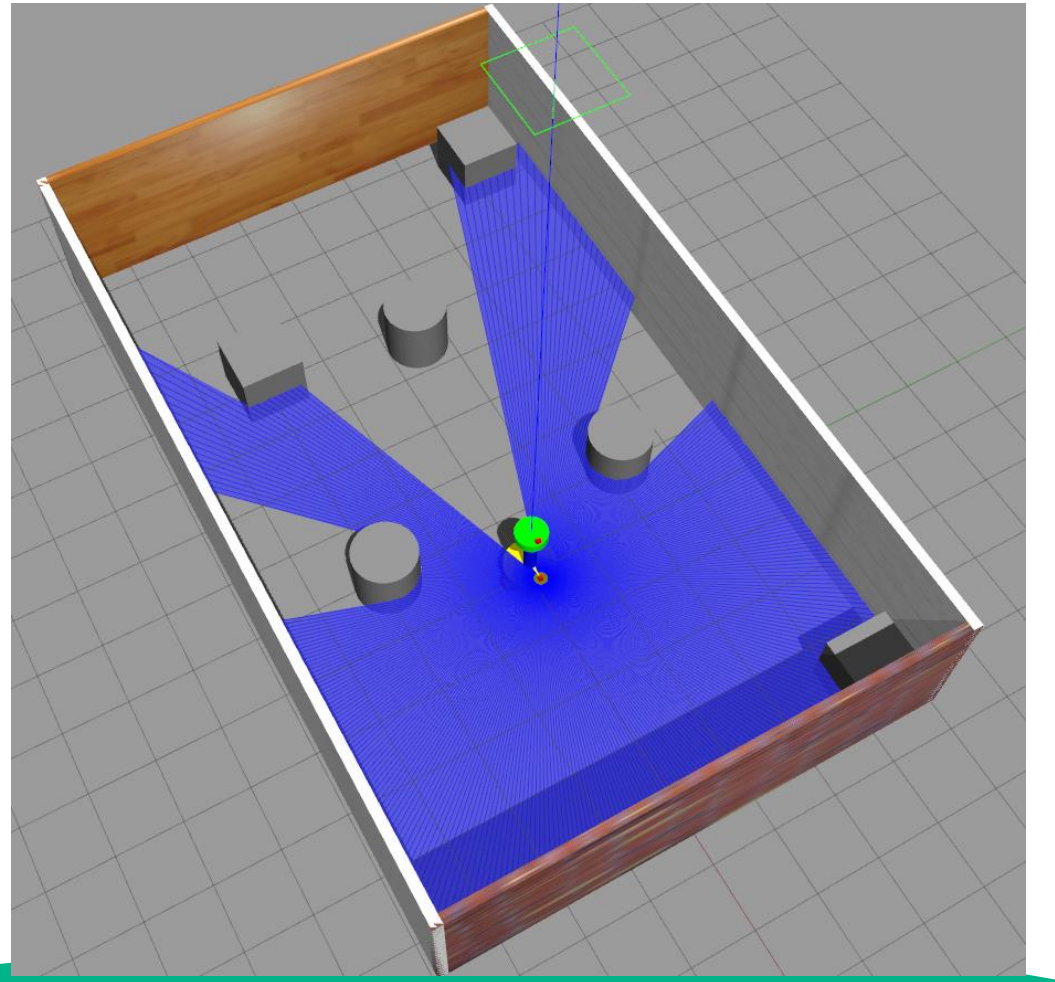
- **问题3：两个驱动轮无法绕 y 轴转动**

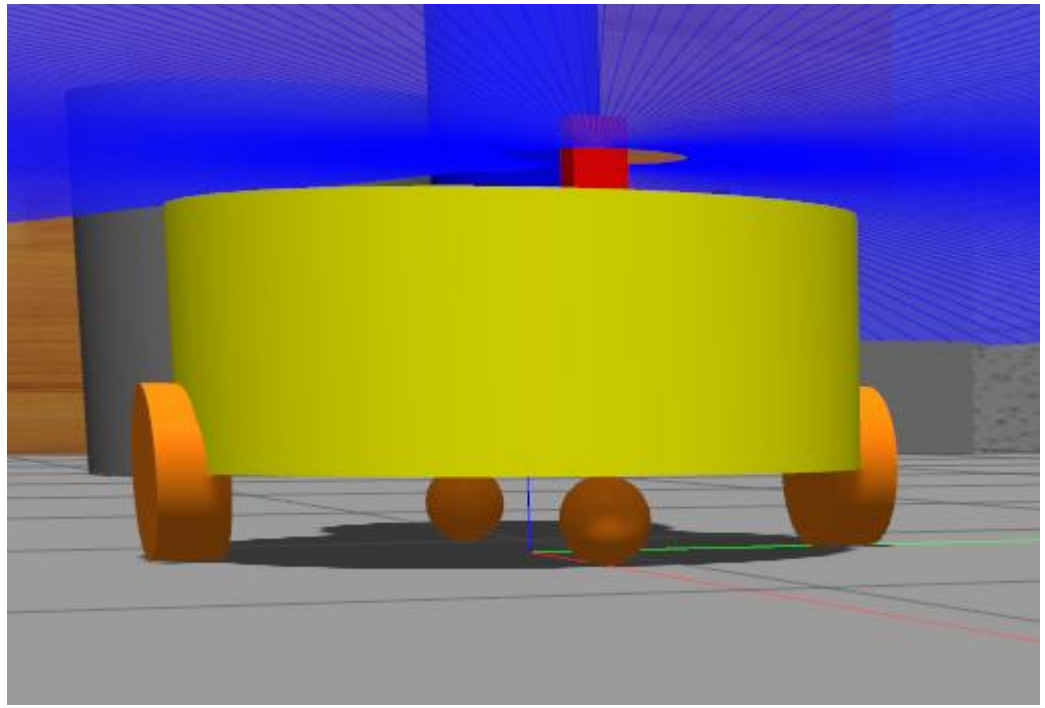
```
<joint name="${wheel_name}_wheel_link2base_link" type="continuous">
    <parent link="base_link"/>
    <child link="${wheel_name}_wheel_link" />
    <origin xyz="0 ${flag * base_radius} ${-base_height / 2}" rpy="0 0 0" />
    <axis xyz="0 1 0" />
</joint>
```

- **问题4：模型倾倒**
  - ✓ **重新设置 link 质量**
  - ✓ **删除 root_link 的 inertial 标签**

1.2 Xacro 模型优化

1.2.1 myrobot_gazebo.xacro







2

Gazebo 控制小车

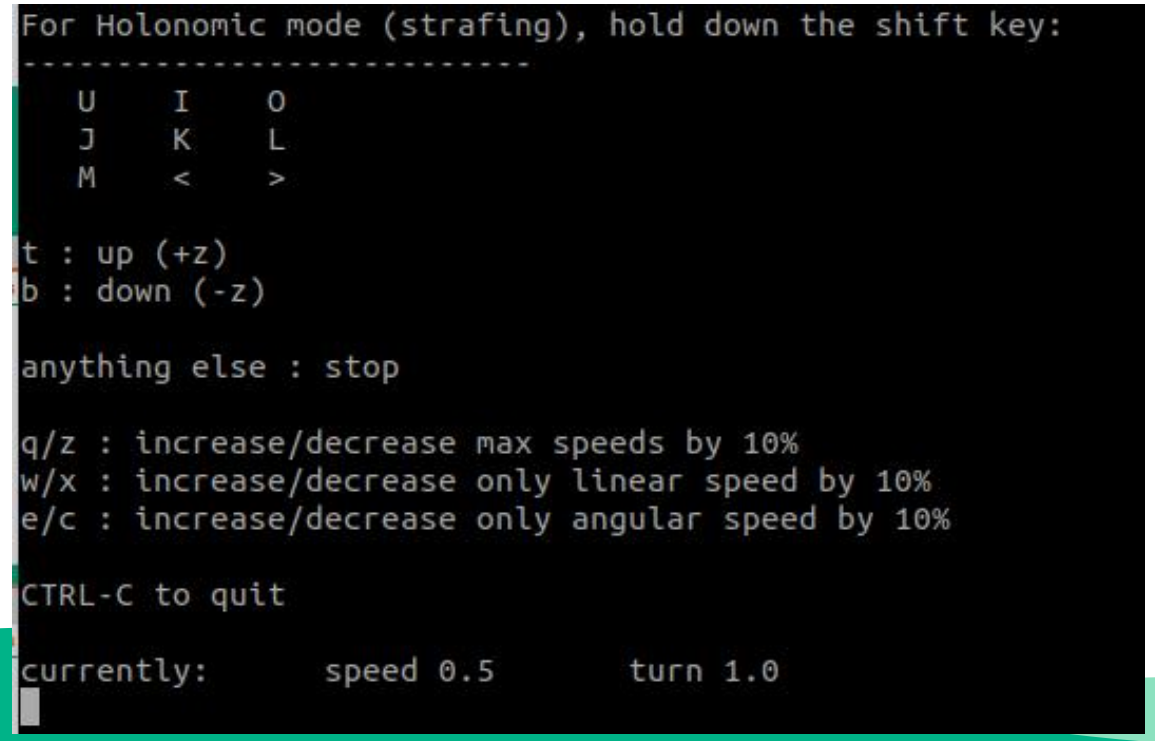
2.1 键盘控制小车运动

2.1.1 teleop_twist_key 键盘控制节点

- 安装和启动

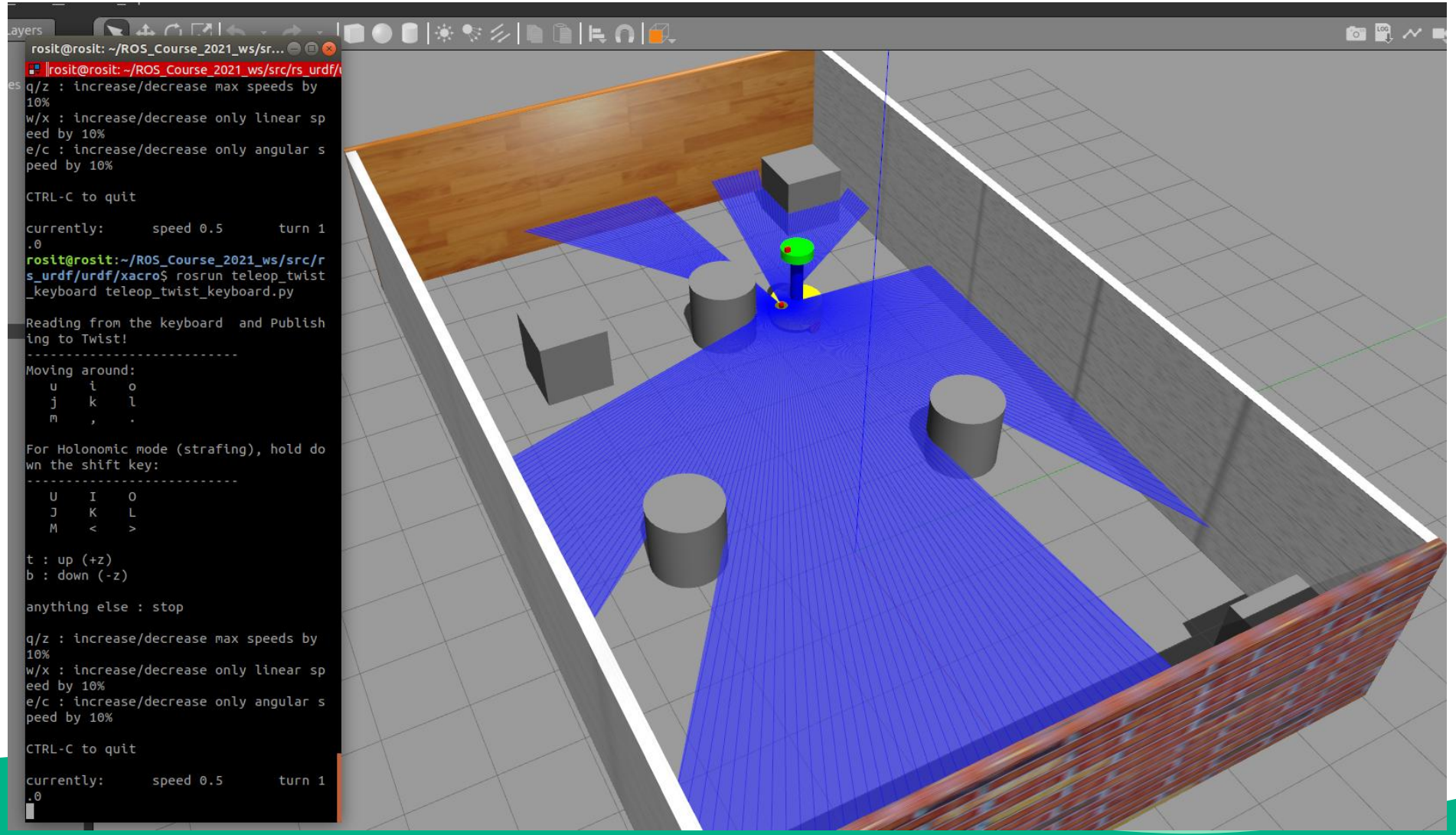
```
$ sudo apt install ros-noetic-teleop-twist-keyboard
```

```
$ rosrun teleop_twist_keyboard teleop_twist_keyboard.py
```



2.1 键盘控制小车运动

2.1.1 teleop_twist_key 键盘控制节点

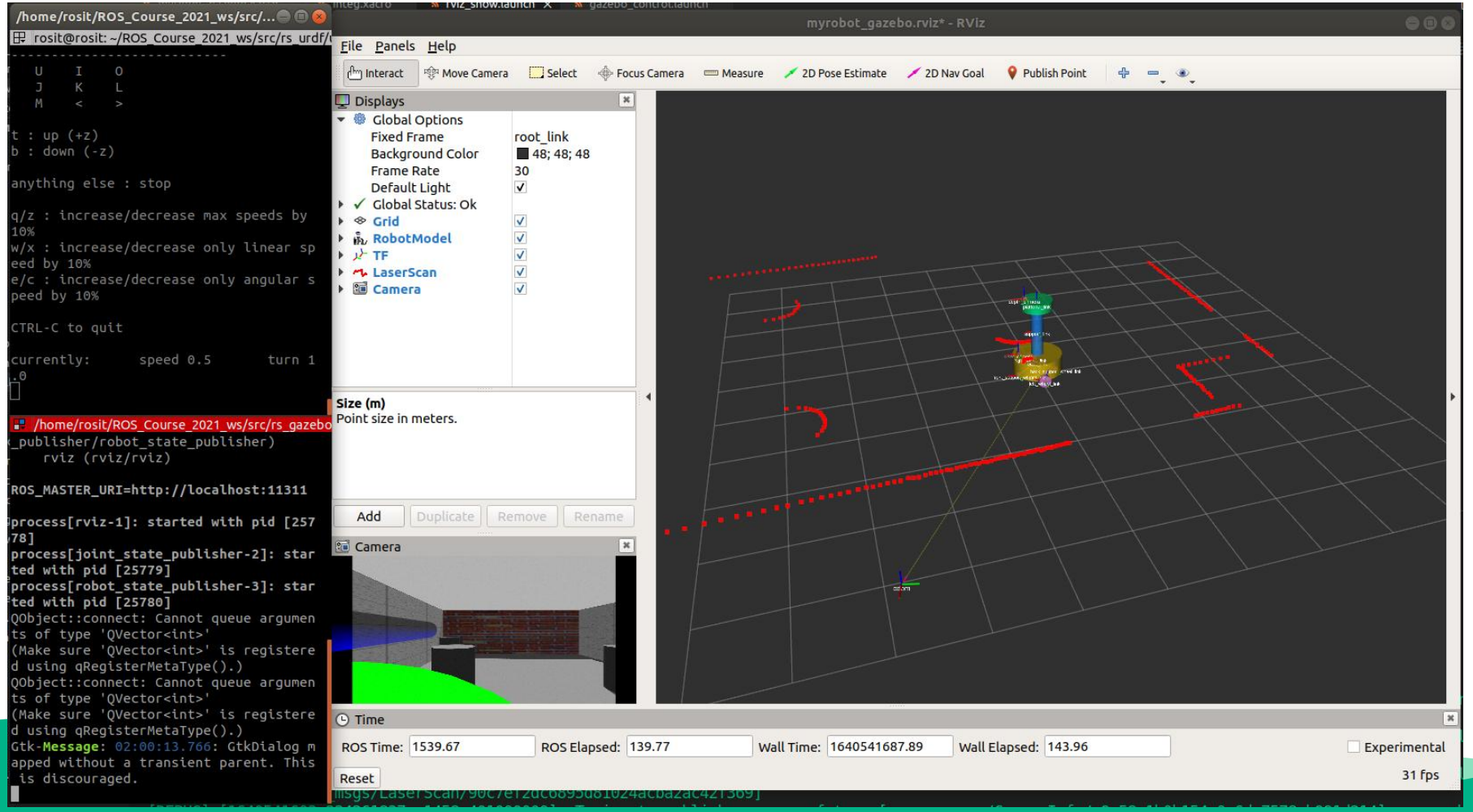


2.1 键盘控制小车运动

2.1.2 rviz 查看激光雷达信息

➢ 新开一个终端，输入以下指令：

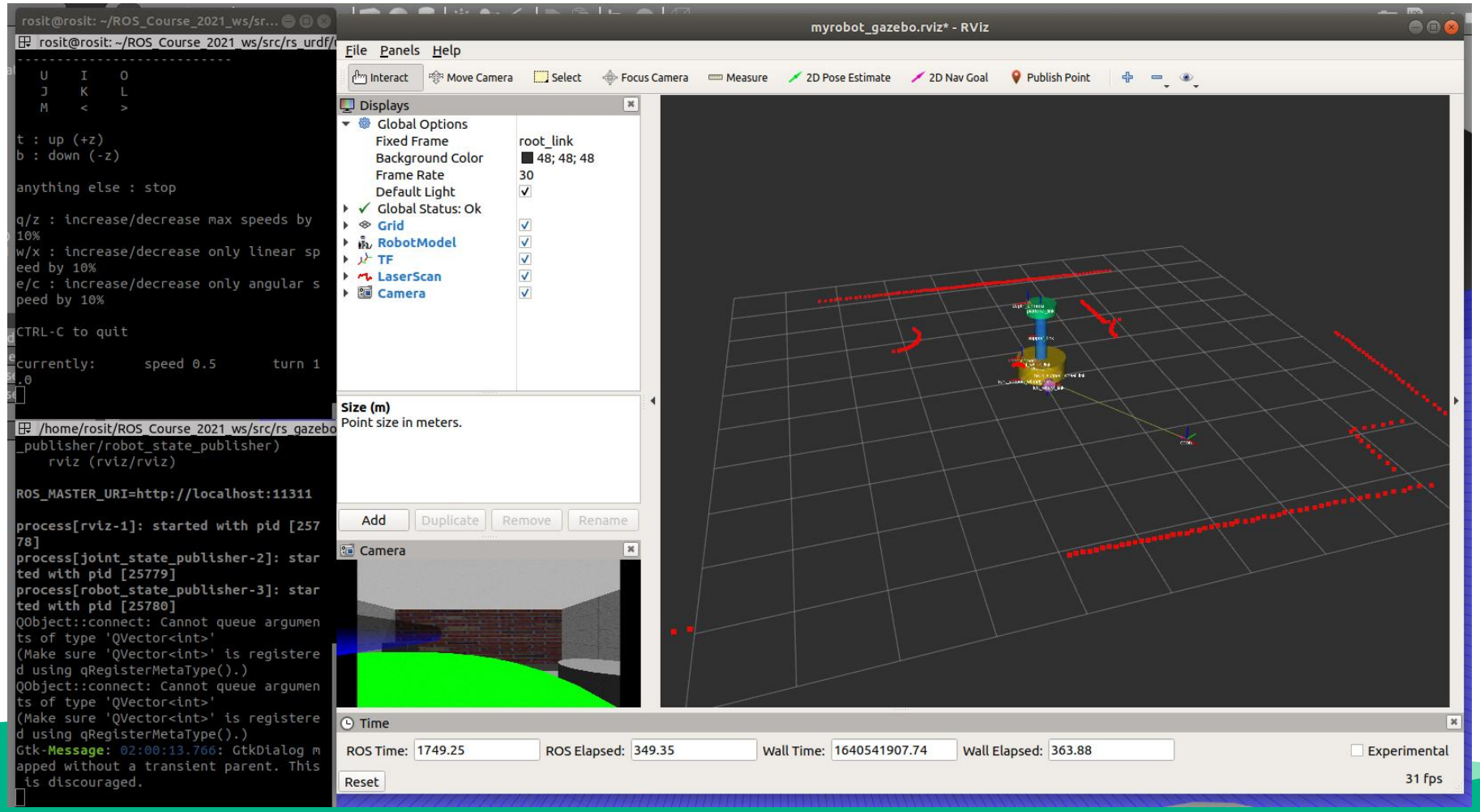
```
$ roslaunch rs_gazebo rviz_show.launch
```



2.1 键盘控制小车运动

2.1.2 rviz 查看激光雷达信息

- 通过键盘控制小车运动，查看激光雷达信息的变化



2.1 键盘控制小车运动

2.1.2 rviz 查看激光雷达信息

- 要求1：修改激光雷达的配置信息，查看各个参数的作用效果

src > rs_gazebo > urdf > gazebo > laser.xacro

```
<robot name="my_sensors" xmlns:xacro="http://wiki.ros.org/xacro">

  <gazebo reference="laser_camera">
    <sensor type="ray" name="rplidar">
      <pose>0 0 0 0 0 0</pose>
      <visualize>true</visualize>
      <update_rate>5.5</update_rate>
      <ray>
        <scan>
          <horizontal>
            <samples>360</samples>
            <resolution>1</resolution>
            <min_angle>-3</min_angle>
            <max_angle>3</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.10</min>
          <max>30.0</max>
          <resolution>0.01</resolution>
        </range>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.01</stddev>
        </noise>
      </ray>
      <plugin name="gazebo_rplidar" filename="libgazebo_ros_laser.so">
        <topicName>/scan</topicName>
        <frameName>laser_camera</frameName>
      </plugin>
    </sensor>
  </gazebo>

</robot>
```

- 参考资料：282-综合应用-雷达仿真

https://www.bilibili.com/video/BV1Ci4y1L7ZZ?p=284

2.1 键盘控制小车运动

2.1.3 rviz 查看摄像头信息

- 要求2：修改摄像头的配置信息，查看各个参数的作用效果

```
<gazebo reference="depth_camera">

  <sensor type="camera" name="camera_node">
    <update_rate>30.0</update_rate>

    <camera name="head">
      <horizontal_fov>1.3962634</horizontal_fov>
      <image>
        <width>1280</width>
        <height>720</height>
        <format>R8G8B8</format>
      </image>
      <clip>
        <near>0.02</near>
        <far>300</far>
      </clip>
      <noise>
        <type>gaussian</type>
        <mean>0.0</mean>
        <stddev>0.007</stddev>
      </noise>
    </camera>

    <plugin name="gazebo_camera" filename="libgazebo_ros_camera.so">
      <alwaysOn>true</alwaysOn>
      <updateRate>0.0</updateRate>
      <cameraName>/depth_camera</cameraName>
      <imageTopicName>image_raw</imageTopicName>
      <cameraInfoTopicName>camera_info</cameraInfoTopicName>
      <frameName>depth_camera</frameName>
      <hackBaseline>0.07</hackBaseline>
      <distortionK1>0.0</distortionK1>
      <distortionK2>0.0</distortionK2>
      <distortionK3>0.0</distortionK3>
      <distortionT1>0.0</distortionT1>
      <distortionT2>0.0</distortionT2>
```

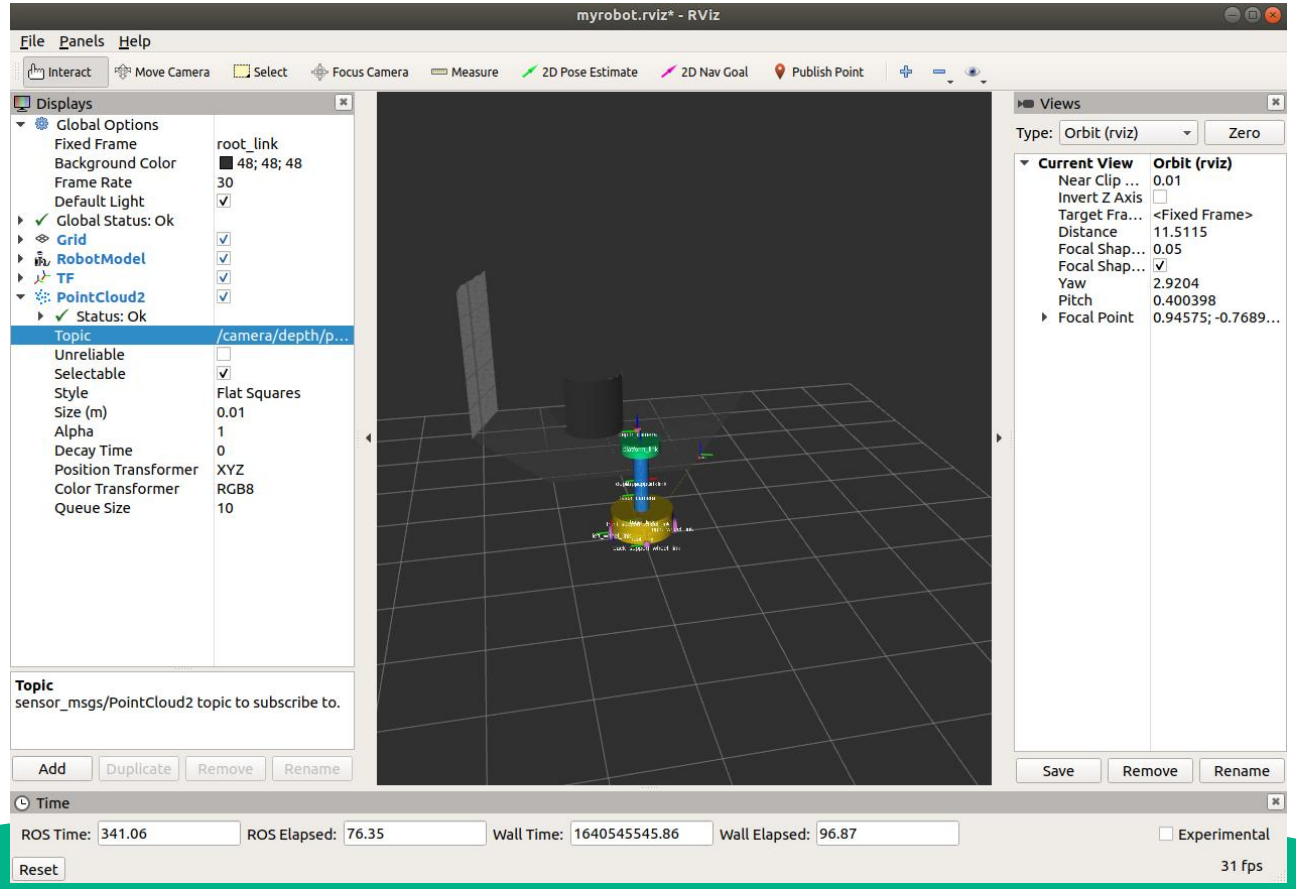
- 参考资料：283-综合应用-摄像头仿真

https://www.bilibili.com/video/BV1Ci4y1L7ZZ?p=285

2.1 键盘控制小车运动

2.1.4 rviz 查看 kinect 信息

- 要求3：添加 kinect 的 xacro 文件，增加 kinect 相机，在 rviz 中查看 kinect 点云信息；修改kinect 的配置信息，查看各个参数的作用效果



- 参考资料：284-综合应用-深度相机仿真-点云

https://www.bilibili.com/video/BV1Ci4y1L7ZZ?p=287

2.1 键盘控制小车运动

2.1.4 rviz 查看 kinect 信息

- 要求3：添加 kinect 的 xacro 文件，增加 kinect 相机，在 rviz 中查看 kinect 点云信息；修改kinect 的配置信息，查看各个参数的作用效果

```
rs_gazebo > urdf > gazebo > kinect.xacro
<robot name="my_sensors" xmlns:xacro="http://wiki.ros.org/xacro">
    <gazebo reference="support_link">
      <sensor type="depth" name="camera">
        <always_on>true</always_on>
        <update_rate>20.0</update_rate>
        <camera>
          <horizontal_fov>${60.0*PI/180.0}</horizontal_fov>
          <image>
            <format>R8G8B8</format>
            <width>640</width>
            <height>480</height>
          </image>
          <clip>
            <near>0.05</near>
            <far>8.0</far>
          </clip>
        </camera>
        <plugin name="kinect_camera_controller" filename="libgazebo_ros_openni_kinect.so">
          <cameraName>camera</cameraName>
          <alwaysOn>true</alwaysOn>
          <updateRate>10</updateRate>
          <imageTopicName>rgb/image_raw</imageTopicName>
          <depthImageTopicName>depth/image_raw</depthImageTopicName>
          <pointCloudTopicName>depth/points</pointCloudTopicName>
          <cameraInfoTopicName>rgb/camera_info</cameraInfoTopicName>
          <depthImageCameraInfoTopicName>depth/camera_info</depthImageCameraInfoTopicName>
          <frameName>support_link</frameName>
          <baseline>0.1</baseline>
          <distortion_k1>0.0</distortion_k1>
          <distortion_k2>0.0</distortion_k2>
          <distortion_k3>0.0</distortion_k3>
          <distortion_t1>0.0</distortion_t1>
          <distortion_t2>0.0</distortion_t2>
          <pointCloudCutoff>0.4</pointCloudCutoff>
        </plugin>
      </sensor>
    </gazebo>
```

- 参考资料：284-综合应用-深度相机仿真-点云

https://www.bilibili.com/video/BV1Ci4y1L7ZZ?p=287



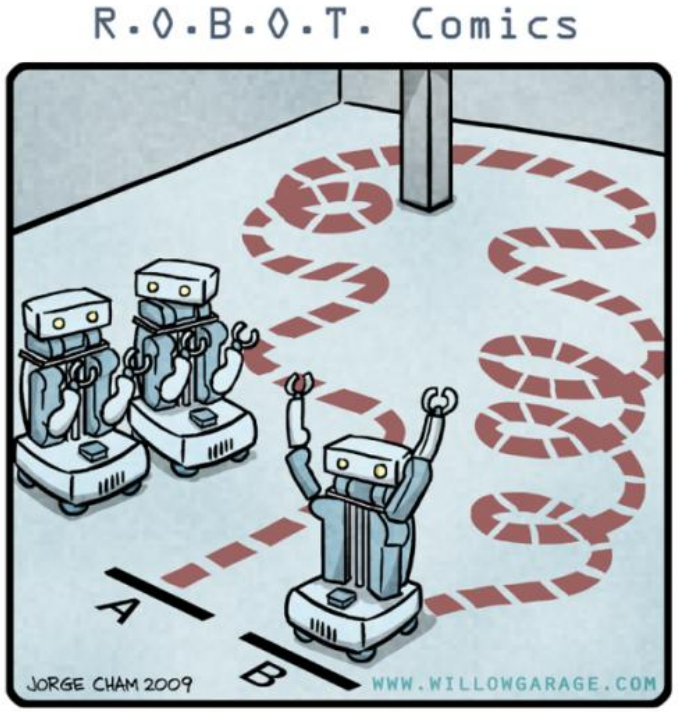
3

自主导航

3.1 Navigation

3.1.1 Overview

- ROS机器人导航(Navigation)由多个功能包组合实现，称为导航功能包集
- 官方介绍：一个二维导航堆栈，它接收来自里程计、传感器流和目标姿态的信息，并输出发送到移动底盘的安全速度命令



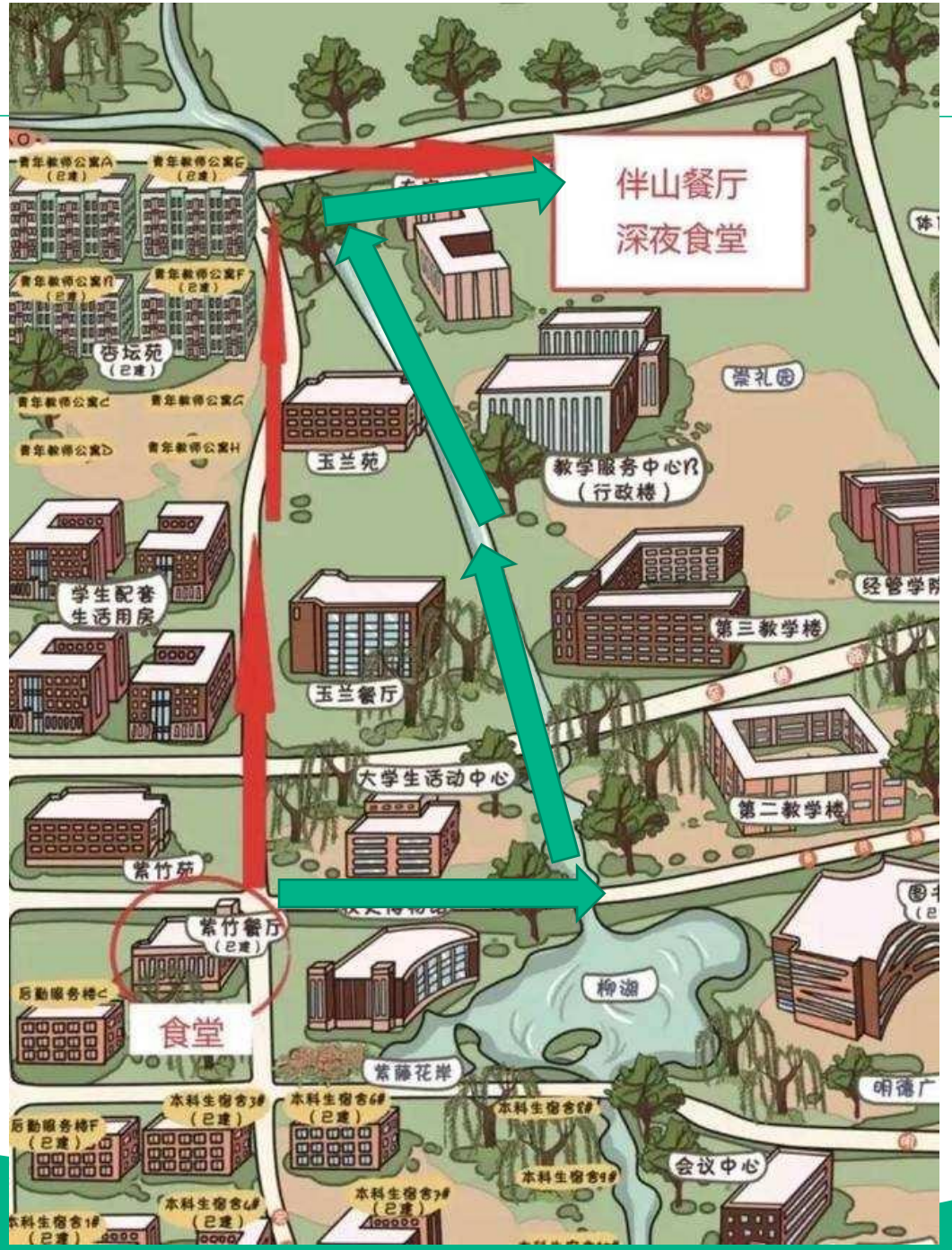
3.1 Navigation

3.1.2 Campus Cases

1. 校园地图(熟悉)
2. 自身位置(结合周围建筑信息)
3. 行走路线

最近的路：从 紫竹餐厅 往北走，看到 玉兰餐厅 再往前走，走到 青年教师公寓E 的那个路口，右拐，过了 专家公寓 就是

走过来就可以，骑自行车也行~



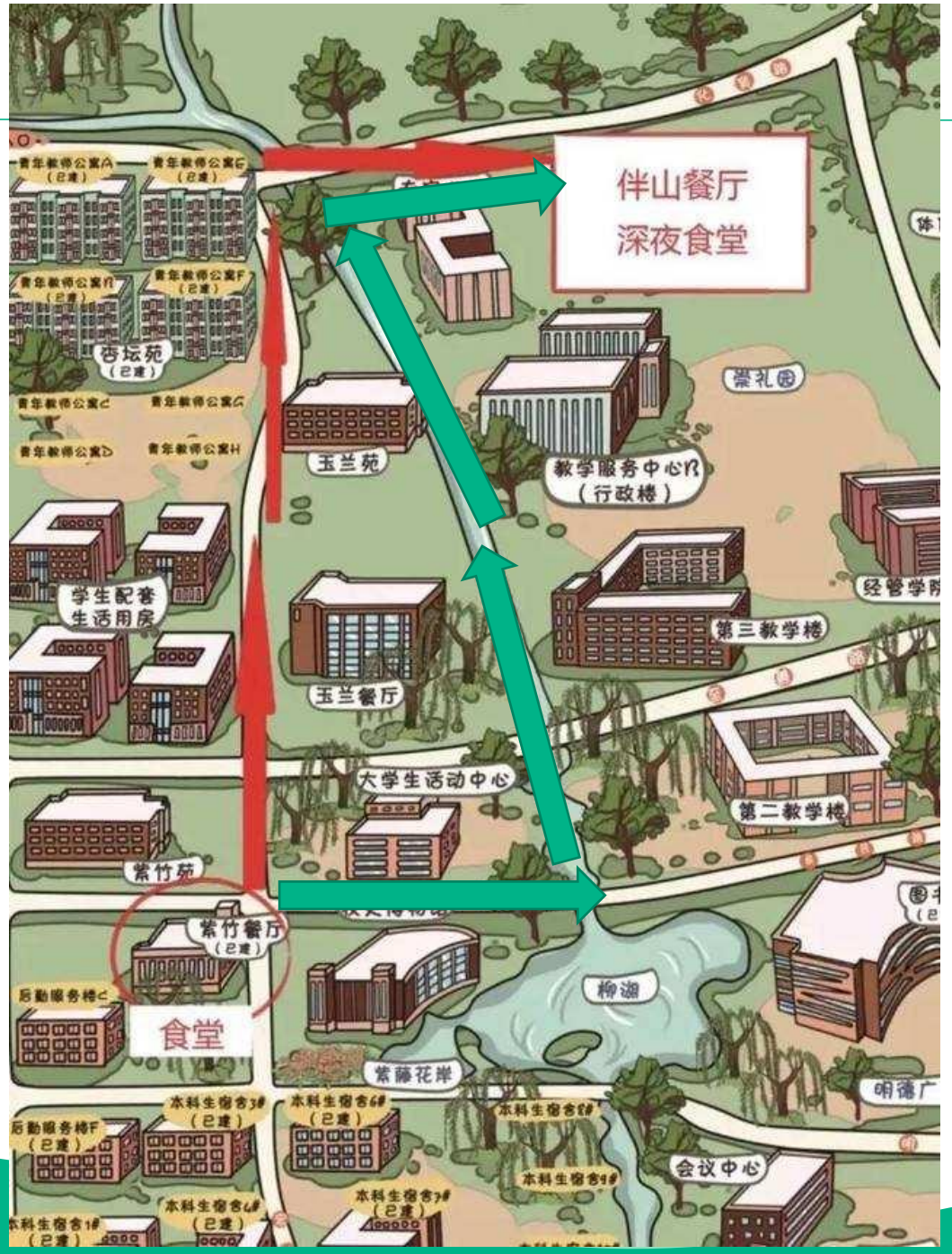
3.1 Navigation

3.1.2 Campus Cases

1. 校园地图(不熟悉)
2. 自身位置(无需建筑信息)
3. 行走路线

最近的路：从 紫竹餐厅 往北走100米，看到一个十字路口，继续 往北走300米，看到一个丁字路口，右转，直行300米就到了

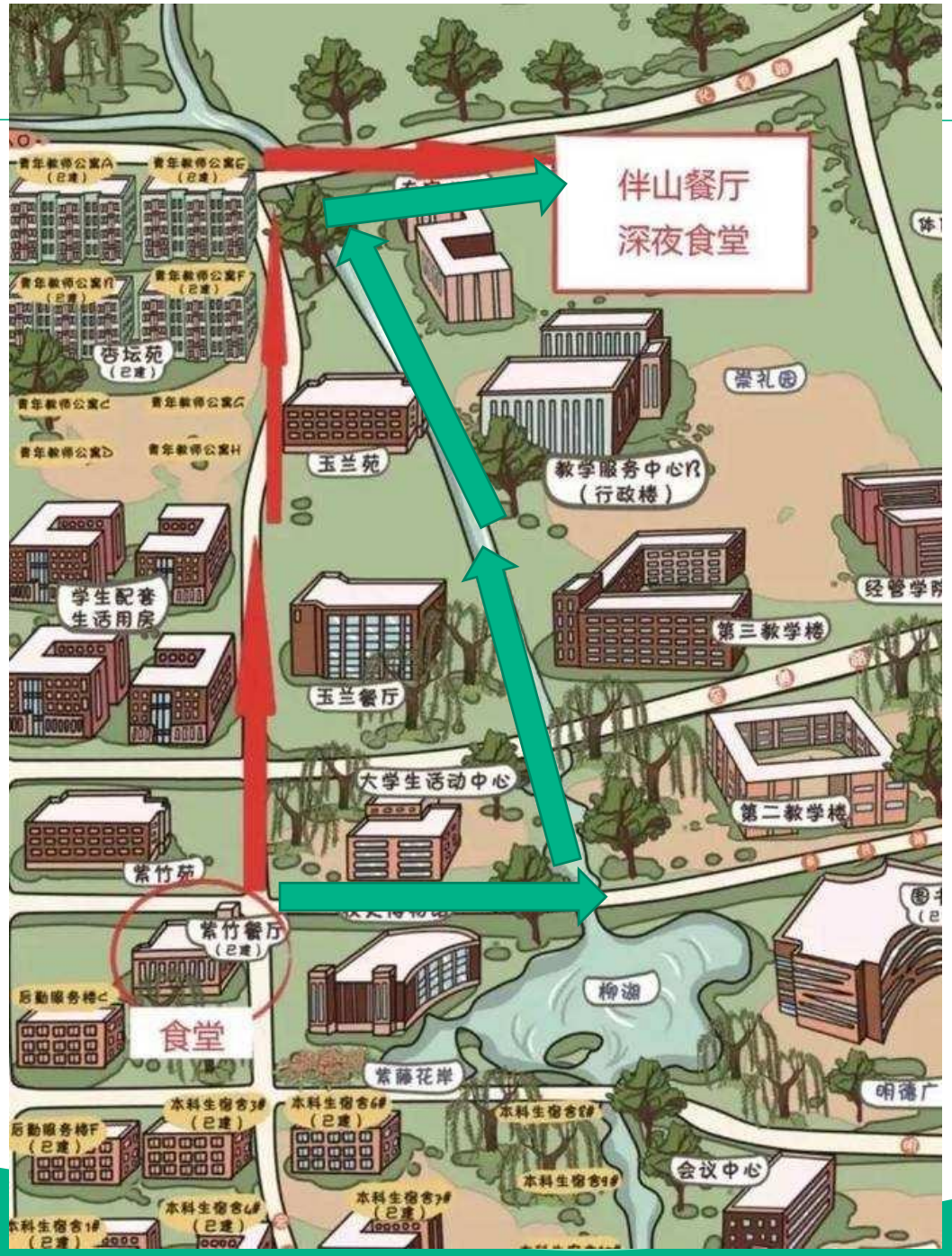
走过来就可以，骑自行车也行~



3.1 Navigation

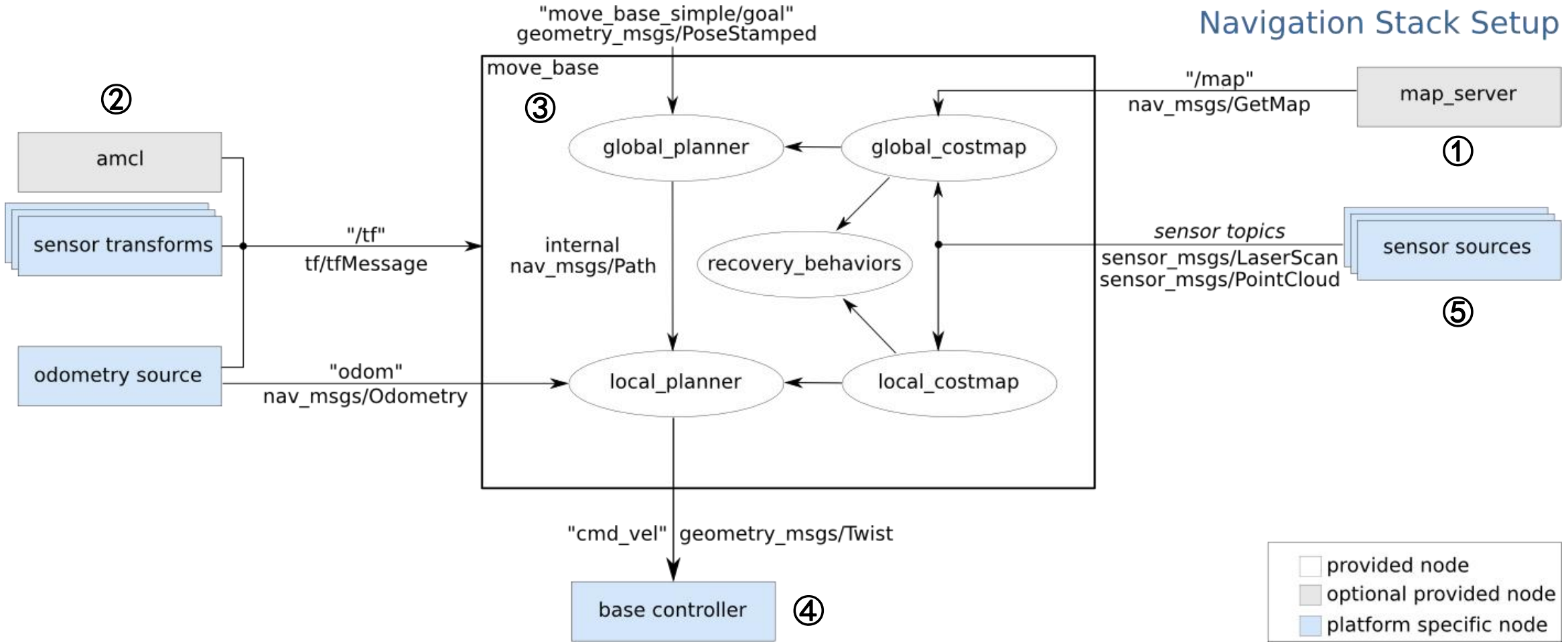
3.1.2 Campus Cases

1. 校园地图 ----------> 全局地图
2. 自身位置 ----------> 自身定位
3. 最近的路 ----------> 路径规划
4. 骑车or走路 ----------> 运动方式
5. 骑车or走路速度 ----------> 运动控制



3.1 Navigation

3.1.3 Navigation structure



1. 全局地图 2. 自身定位 3. 路径规划 4. 运动控制 5.环境感知

3.1 Navigation

3.1.3 Navigation structure

1 全局地图

- SLAM(simultaneous localization and mapping)，即时定位与地图构建。SLAM问题可以描述为: 机器人在未知环境中从一个未知位置开始移动，在移动过程中根据位置估计和地图进行自身定位，同时在自身定位的基础上建造增量式地图，以绘制出外部环境的完全地图

- 在 ROS 中，常用的 SLAM 包括 gmapping、hector_slam、cartographer、rgbdslam、ORB_SLAM等

- SLAM建图 ，机器人需具备感知外界环境的能力，尤其是获取周围环境深度信息的能力。感知的实现需要依赖于传感器，比如: 激光雷达、摄像头、RGB-D摄像头等

- SLAM 生成的地图需要被保存以待后续使用，在 ROS 中保存地图的功能包是 map_server

3.1.3 Navigation structure

2 自身定位

- 导航过程中，机器人需要实时确定当前自身的位置。如果在室外，可以使用GPS辅助定位；在室内、隧道、地下或一些屏蔽 GPS 信号的区域，需要使用具备自身定位功能的功能包，例如：AMCL
- AMCL(adaptive MonteCarlo Localization)自适应的蒙特卡洛定位，是用于2D移动机器人的概率定位系统。它实现了自适应（或KLD采样）蒙特卡洛定位方法，该方法使用粒子过滤器根据已知地图跟踪机器人的姿态

3.1.3 Navigation structure

3 路径规划

- 导航过程中，机器人需要根据目标位置计算最优路径，此过程称为全局路径规划。在运动过程中，机器人需要根据障碍物动态调整运动路线，此过程就称为局部路径规划

- 在 ROS 中提供了 move_base 功能包来实现路径规则，该功能包主要由两大规划器组成

  - 1 全局路径规划(gloable_planner)

    根据给定的目标点和全局地图实现总体的路径规划，使用 A* 或 Dijkstra 算法进行全局路径规划，计算最优路线，作为全局路线

  - 2 局部路径规划(local_planner)

    局部路径规划的作用就是使用相应算法(Dynamic Window Approaches，DWA) 等来实现障碍物的规避，并选取当前最优路径以尽量符合全局最优路径

3.1.3 Navigation structure

4. 运动控制

- move_base功能包通过话题"cmd_vel"发布geometry_msgs/Twist类型的消息，这个消息基于机器人的基座坐标系，它传递的是运动命令
- 运动节点订阅"cmd_vel"话题， 将该话题上的速度命令转换为电机命令并进行运动控制

3.1.3 Navigation structure

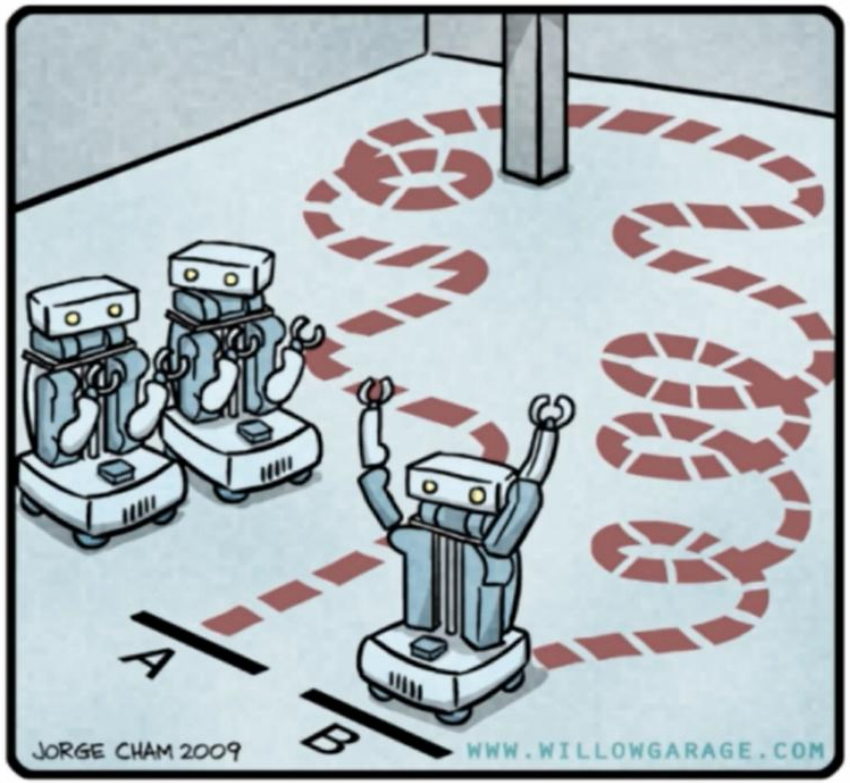
5. 环境感知

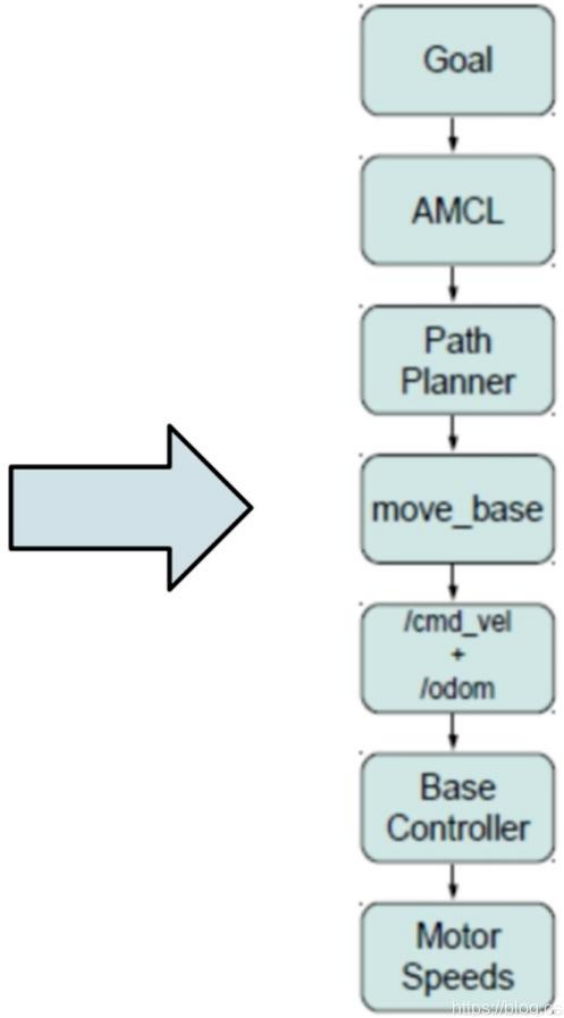
- **感知周围环境信息，比如: 摄像头、激光雷达、编码器等**
- **摄像头、激光雷达可以用于感知外界环境的深度信息，编码器可以感知电机的转速信息，通过获取速度信息并生成里程计信息**

在导航功能包集中，环境感知是重要模块，它为其他模块提供了支持，包括 SLAM、AMCL、move_base 都需要依赖于环境感知

3.1 Navigation

3.1.3 Navigation structure





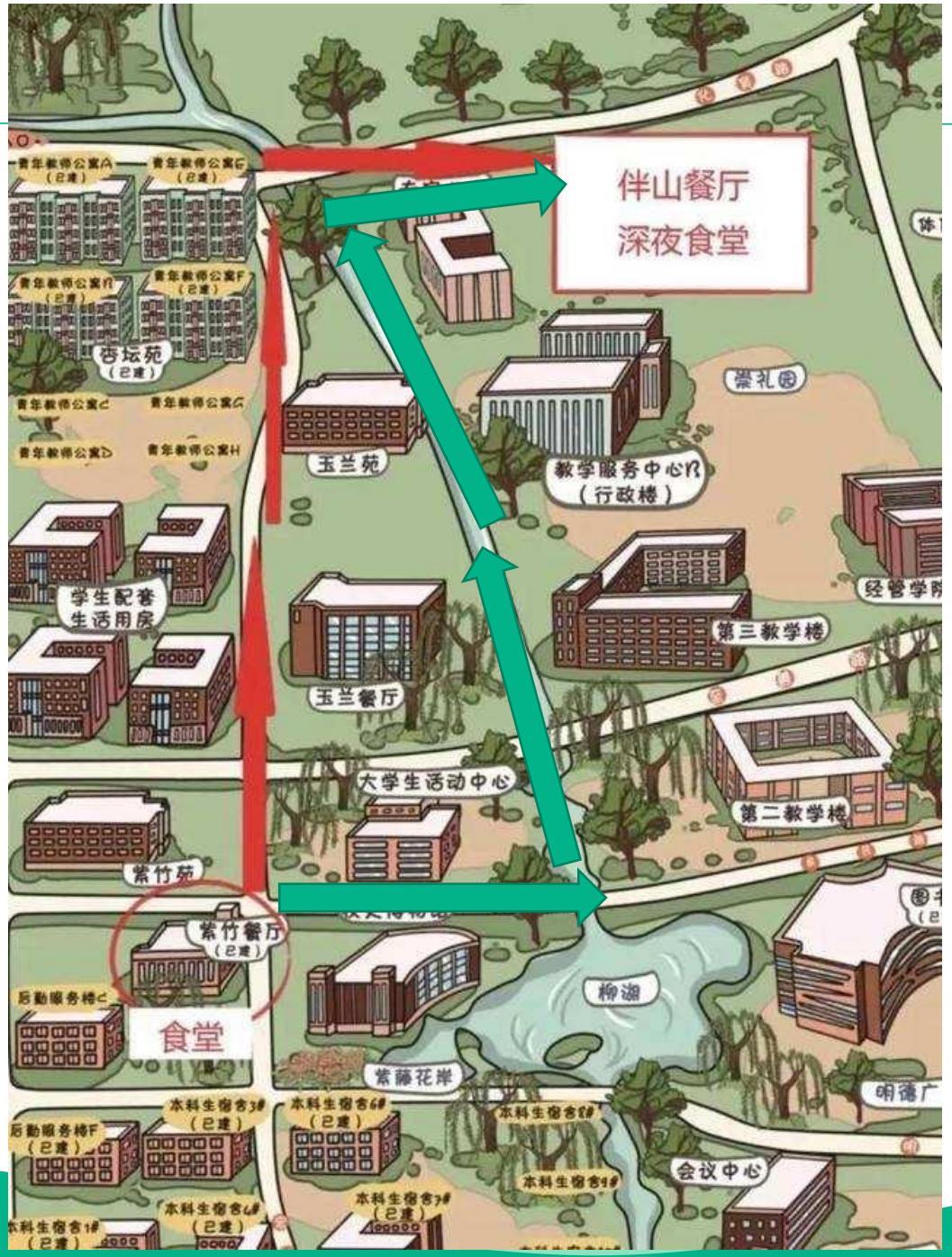
3.1 Navigation

3.1.4 Odom

定位实现方式有两种：

1 通过里程计定位：实时收集机器人的速度信息，计算并发布机器人坐标系与父级参考系的相对关系

2 通过传感器定位：通过传感器收集外界环境信息，通过匹配计算并发布机器人坐标系与父级参考系的相对关系



3.1 Navigation

3.1.4 Odom

两种定位方式优缺点：

1 里程计定位

优点：里程计定位信息是连续的，没有离散的跳跃

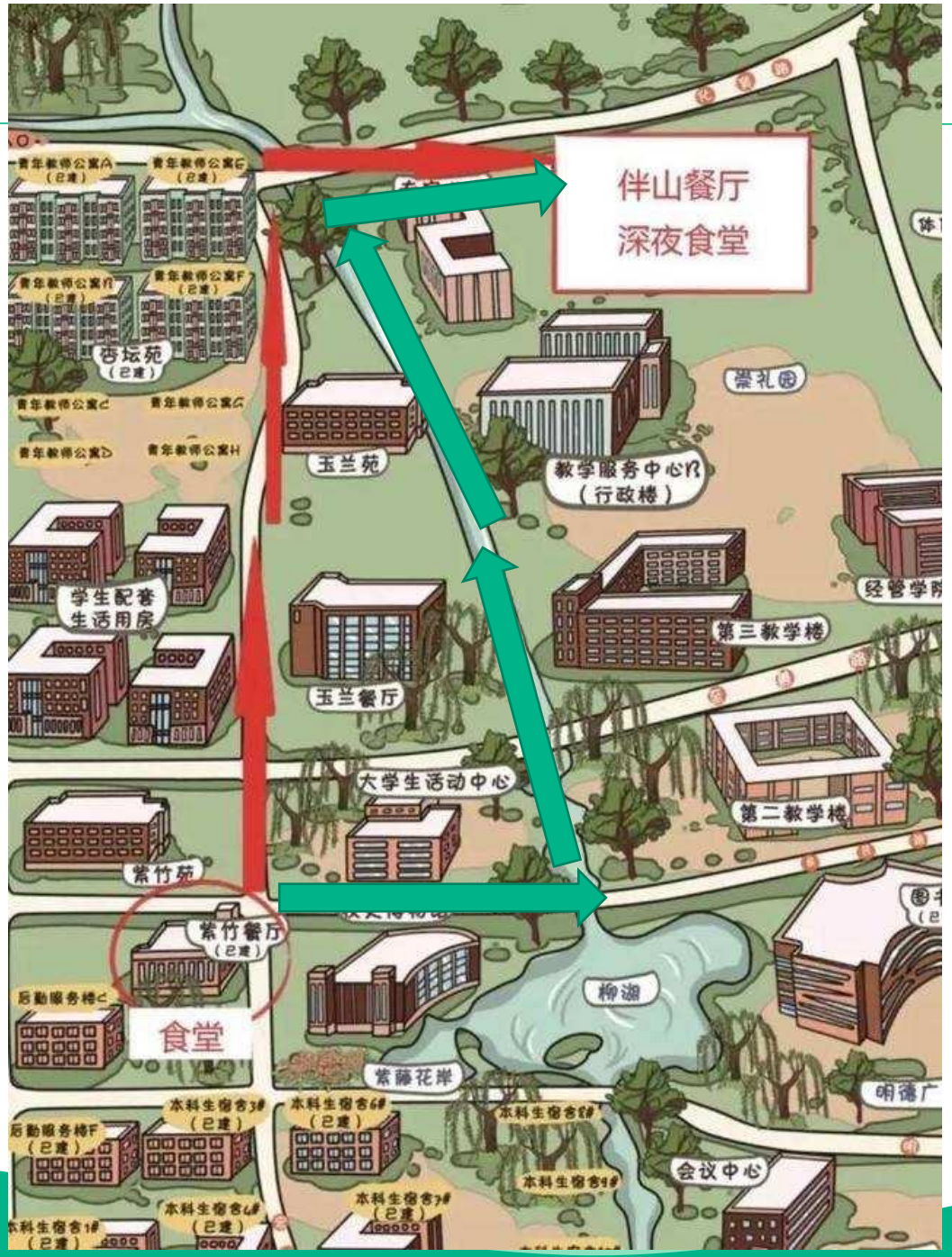
缺点：里程计存在累计误差，不利于长距离或长期定位

2 传感器定位

优点：比里程计定位更精准

缺点：传感器定位会出现跳变的情况，在标志物较少的环境下，其定位精度会大打折扣

两种定位方式优缺点互补，应用时一般二者结合使用



3.1.4 Odom

- 上述两种定位实现中，机器人坐标系一般使用机器人模型中的根坐标系(root_link)
- 里程计定位时，父级坐标系一般称之为 odom
- 如果通过传感器定位，父级参考系一般称之为 map
- 当二者结合使用时，map 和 odom 都是机器人模型根坐标系的父级，一般会将转换关系设置为: map -> doom -> base_link 或 root_link

3.1 Navigation

3.1.5 ROS packages

1 安装相关的ROS功能包：

gmapping (用于构建地图)

```
$ sudo apt install ros-noetic-gmapping
```

地图服务map-server(用于保存与读取地图)

```
$ sudo apt install ros-noetic-map-server
```

navigation (用于定位以及路径规划)

```
$ sudo apt install ros-noetic-navigation
```

2 使用方法：

新建功能包，并导入依赖: gmapping map_server amcl move_base

3.1 Navigation

3.1.5 ROS packages

1.先启动 Gazebo 仿真环境 launch 文件

```
$ roslaunch rs_nav gazebo_control.launch
```

2.然后再启动地图绘制的 launch 文件

```
$ roslaunch rs_nav gmapping.launch
```

3.启动键盘键盘控制节点，用于控制机器人运动建图

```
$ rosrun teleop_twist_keyboard teleop_twist_keyboard.py
```

4.在 rviz 中添加组件，进行导航操作

```
$ roslaunch rs_nav move_base.launch
```

Q&A

今日事今日毕

Question?